RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

Skin Disease Detection System

MLOps Tutorial Report

submitted by

Shashank Krishnamani	**1RV23AI091**
Shalini P	**1RV22AI088**
Shravyaa S	**1RV23AI093**
Shanthesh A S	**1RV23AI090**

Artificial Intelligence and Machine Learning

2025-2026

TABLE OF CONTENTS

Phase 1:Model Development and Foundations		
Chapter 1		
1.1	**Problem Statement**	
1.2	**Objectives**	
1.3	**Project Scope**	
1.4	**Evaluation Metrics**	
1.5	**Tools Used**	
1.6	**Environment Setup**	
1.7	**Literature Review**	
Chapter 2		
2.1	**Understanding MLOps, Risks and Requirement Analysis**	
2.2	**Risk Matrix**	
Chapter 3		
3.1	**Data Collection and Exploration Techniques**	
3.2	**Data Profiling, cleaning and preprocessing**	
3.3	**Feature Extraction and Feature Selection**	
3.4	**Model Design Approaches**	
3.5	**Governance and Data Version Control**	
Chapter 4		
4.1	**Model Building**	
4.2	**Training, Validation and Testing**	
4.3	**Evaluation Metrics and Visualisation**	

4.4	Experiment Tracking	
4.5	Interpreting Model Results and Error Analysis	
Chapter 5		
5.1	Slides	
Phase 2:Deployment, Monitoring and Governance		
Chapter 6		
6.1	Revisiting Model Performance	
6.2	Hyperparameter Tuning and Performance	
6.3	Interpretability	
6.4	Model Versioning and Registry	
6.5	Documentation of Model Updates	
Chapter 7		
7.1	Overview of CI/CD for ML	
7.2	Tools Setup:GitHub Actions/Jenkins/GitLab CI	
7.3	Automating Model Training and Testing	
7.4	Building and Running a Deployment Pipeline	
7.5	Continuous Integration Hands-On Exercise	
Chapter 8		
8.1	Model Monitoring Concepts and KPI's	
8.2	Tools: Prometheus, Grafana, EvidentlyAI	
8.3	Detecting Concept Drift and Data Drift	

8.4	**Model Retraining and Redeployment**	
8.5	**Logging,Alerting and Dashboard Setup**	
Chapter 9		
9.1	**Post-Deployment Performance Review**	
9.2	**Continuous Improvement via Feedback Loops**	
9.3	**Governance and Ethical AI Practices**	
9.4	**Documentation: Model Cards, Data Cards**	
9.5	**Audit and Review Checklist**	

Chapter 1

INTRODUCTION

1.1 Problem Definition:

Skin diseases affect a large portion of the global population, yet most deep learning–based skin disease classification models remain limited to experimental environments. Despite high reported accuracies, over 60% of medical ML models fail to be deployed in real-world settings due to poor reproducibility, lack of data and model versioning, and absence of systematic lifecycle management. Additionally, dermatological image datasets are typically imbalanced and heterogeneous, often exhibiting 3–5× class distribution variance, which leads to 10–25% performance degradation when models encounter real-world data drift.

The primary challenge addressed in this work is the lack of a robust MLOps framework that ensures statistical reliability, traceability, and scalability of skin disease classification systems. This project targets the development of a reproducible MLOps pipeline that supports efficient model updates without full deep learning retraining, maintains consistent performance tracking across experiments, and enables continuous evaluation under evolving data distributions.

1.2 Objectives:

- Accessible Screening: To provide a fast and easy-to-use tool for identifying five common skin conditions like Atopic Dermatitis, Eczema, Benign Keratosis, Tinea/Fungal Infections, and Seborrheic Keratoses.
- Bridge Healthcare Gaps: To act as a diagnostic assistant in remote or underserved areas where expert dermatologists are scarce, helping non-specialist workers make informed referrals.
- To continuously safeguard prediction quality for users in real-world settings by monitoring model performance and detecting data drift that could degrade diagnostic reliability over time.

1.3 Project Scope:

- Disease Focus: The project is strictly limited to the detection of five specific skin conditions: Eczema, Atopic Dermatitis, Benign Keratosis-like Lesions, Seborrheic Keratoses, and Tinea/Fungal Infections.
- Data Input: The system accepts standard photographic or dermoscopic images, which are preprocessed and resized to a uniform dimension.
- Technological Boundary: The focus remains on the diagnostic accuracy of the hybrid CNN-Ensemble pipeline, rather than external factors like patient history or genetic data.

1.4 Evaluation Metrics:

- Accuracy: The primary measure of overall correctly classified instances across all five classes.
- Precision & Recall: To ensure the model effectively distinguishes between visually similar diseases without excessive false positives or missed cases.
- F1-Score: A balanced metric that accounts for both precision and recall, especially useful if any class has a different number of sample images.
- Confusion Matrix: A key visualization tool to identify exactly which diseases are being confused with one another.

1.5 Tools used:

Component	**Tool Used**	**Purpose**
Data Processing	• Numpy • pandas	Used for high-performance numerical operations on image arrays. Helpful in handling and organizing metadata or structured data formats.
Model Development	TensorFlow / Keras	Used to build and train the Convolutional Neural Network (CNN) model for

		image-based feature extraction.
Data Management	DVC + Git	Version control of datasets, ensuring reproducibility and traceability of user data across experiments.
Experiment Tracking and Version Control	ZenML	Tracks pipelines, parameters, metrics, and artifacts across runs.
Automation & Pipeline Orchestration	ZenML	Modular pipeline execution for feature extraction, training, and evaluation.
Model Deployment & Serving	Docker	Ensures environment consistency and reproducible model execution
Monitoring and Governance	Evidently AI	Detects data drift, prediction drift, and performance degradation affecting users
Artifact Storage	MinIO	Object storage backend for ZenML artifacts.

1.6 Environment Setup:

The project environment is configured to ensure reproducibility, scalability, and consistent execution across systems. Development is carried out using Python 3.10 within an isolated virtual environment, with all dependencies managed through a centralized requirements.txt file. Git is used for source code version control, while DVC manages large datasets and experiment reproducibility by tracking data versions separately from the codebase. The machine learning workflow is orchestrated using ZenML, which provides pipeline abstraction, experiment lineage, and artifact tracking. MinIO is employed as the artifact store backend for ZenML to persist models, features, and metadata in a scalable object storage system. To ensure deployment consistency, the

application and pipelines are containerized using Docker, enabling platform-independent execution. Evidently AI is integrated to monitor data quality and detect data drift during model inference, supporting reliable post-deployment model behavior.

1.7 Literature Review:

Mysaa Khaled Alrahal et al. [1] empirically compares open-source MLOps tools in automated pipelines, highlighting the role that data version control and experiment tracking play in reproducibility and scalability. It demonstrates that tools like DVC significantly enhance traceability and pipeline automation, reducing manual error and enabling efficient experimentation across both text- and image-based ML tasks. This makes it relevant to your use of DVC for dataset versioning and model lifecycle management.

Sambasivarao et al. [2] analyzes machine learning, deep learning, and transfer learning for automated skin disease classification, noting the performance trade-offs between traditional classifiers and deep CNN/transfer methods such as ResNet and VGG. It highlights the importance of pre-trained models and transfer learning in improving accuracy on limited dermatological datasets and discusses challenges including data standardization and algorithmic bias.

Beyza Eken et al. [3], this review consolidates 150 studies to analyze MLOps practices and challenges. It emphasizes the need for toolchains integrating DVC, ZenML, and monitoring tools to support production ML, noting interoperability and standardization issues.

Jayanth Mohan et al [4] explores advanced vision transformer (ViT) architectures (e.g., Swin, DinoV2) for multi-class skin disease classification, demonstrating that transformer models fine-tuned with ImageNet weights can outperform CNN benchmarks on large heterogeneous datasets. Explainable AI techniques such as GradCAM and SHAP are used to visualize model predictions and aid clinical interpretability, bridging performance with explainability.

Berberi et al. [5], this paper surveys the MLOps tooling ecosystem and highlights how different open-source and commercial tools handle operations such as drift detection, monitoring, orchestration, and automation. It specifically notes that platforms and frameworks like ZenML integrate with drift detection libraries such as Alibi-detect, providing standardized methods for detecting data and concept drift, a key operational concern when deploying models. This work places tools like ZenML and Evidently within the broader landscape of ML lifecycle support and shows how they contribute to observability and continuous monitoring.

Faezeh Amou Najafabadi et al. [6], this mapping study categorizes tools by lifecycle role—data versioning, orchestration, monitoring, and model management. It shows how DVC handles versioning while ZenML and Alibi-detect support orchestration and monitoring for coherent MLOps stacks.

Mallardi et al. [7] propose an end-to-end MLOps framework for deploying ML models in healthcare systems, emphasizing automation, reproducibility, and lifecycle governance. The study adapts DevOps practices to ML workflows to ensure reliable deployment and monitoring in regulated environments. Their work highlights artifact traceability and pipeline standardization for clinical integration. Tools discussed include Docker for containerization, CI/CD pipelines, and workflow orchestration frameworks.

Furqon et al. [8] proposes an Android application utilizing (CNN) with MobileNetV2 architecture to detect eight common skin diseases. This architecture was selected for its efficiency and high accuracy, making it suitable for mobile implementations. The system aims to address the challenges of skin health, particularly in regions like Indonesia with a high prevalence of such conditions. The model achieved a 97% accuracy on the test dataset, with the Android application demonstrating 84% accuracy on general data.

Sitcheu et al. [9] presents an MLOps pipeline tailored for data-scarce biomedical imaging, addressing challenges of model drift and limited training samples. The authors emphasize continuous deployment and monitoring rather than one-time training. Dataset versioning and model fingerprinting are used to manage evolving data distributions. Tools include dataset versioning systems, ML pipeline orchestrators, and monitoring dashboards.

R. sadik et al. [10] leverages CNN architectures, MobileNet and Xception, combined with transfer learning and augmentation techniques for efficient skin disease diagnosis. It addresses the challenges of visual similarities between dermatological conditions by proposing an expert system. This system aims to accurately recognize five common skin disorders: Atopic dermatitis, Eczema, Herpes, Nevus, and Melanoma, achieving high classification accuracies of 96% and 97% respectively.

Agarwal et al. [11] proposes a CNN-based model to automatically classify seven types of skin lesions using the HAM10000 dataset. The model architecture includes convolutional layers and batch normalization to enhance training stability and improve convergence. The Adam optimizer is employed for efficient learning.This system aims to support early and accurate diagnosis of skin conditions.

Patel et al. [12], consolidates various machine learning approaches for skin disease classification, outlining how algorithms such as SVM, Random Forest, and ensemble methods have been applied across datasets. It provides a broad overview of feature extraction techniques, classifier performance, and common preprocessing practices, forming a baseline comparison for both traditional and deep learning approaches.

Kreuzberger et al. [13] provide a comprehensive overview of MLOps by defining its core principles, architectural components, and operational roles to bridge the gap between ML research and production. The paper adopts a mixed-method approach to formalize how DevOps practices such as CI/CD, automation, and workflow orchestration are adapted for ML systems. It emphasizes

end-to-end operationalization, including artifact versioning, deployment automation, and continuous evaluation. The study references tools such as Git/GitHub, Jenkins/GitLab CI, Docker/Kubernetes, and monitoring frameworks like Prometheus for scalable and reproducible ML operations.

Shetty et al. [14] presents a deep learning approach for classifying eight skin diseases using CNNs. A dataset of 25,000+ images was used with preprocessing and transfer learning. 11 pre-trained models were tested; ResNet152 performed best with ~74% accuracy. The study shows CNNs are effective for skin disease classification and suggests future work on larger datasets and newer architectures.

Vasudha Rani et al [15] presents a comparative study of five popular machine learning models like CART, Decision Tree, SVM, Random Forest, and Gradient Boosting for the classification of six different skin diseases. The researchers applied preprocessing techniques to enhance data quality and evaluated each algorithm's performance. Among the individual models, Random Forest delivered the best results with an accuracy of 97.3%. Furthermore, an ensemble approach that combined all five classifiers achieved a significantly higher accuracy of 98.64%.

Hewage and Meedeniya [16], This survey reviews MLOps tools for versioning, automation, deployment, and monitoring. It highlights tools like Git, DVC, and MLflow for reproducibility and experiment tracking, while noting the lack of fully integrated end-to-end platforms.

P. N. Srinivasu et al. [17] combines MobileNet V2 for feature extraction with LSTM classifiers to recognize multiple skin disease types from dermoscopic images, achieving high accuracy with lightweight architectures suitable for real-time or resource-constrained environments. The approach emphasizes the use of pre-trained mobile CNNs and sequential classifiers for improved robustness.

Kassem et al. [18], This systematic review categorizes 100+ studies on machine learning and deep learning for skin lesion analysis, comparing methodologies and reporting common issues like small datasets, racial bias, and segmentation challenges. The review highlights the proliferation of deep learning solutions that leverage data augmentation and ensemble methods to improve classification performance, underscoring barriers to clinical adoption.

Naronqlerdrit and Mporas [19] evaluates pre-trained CNNs, especially ResNet-50, for classifying dermoscopic images. Overall accuracy reaches 93.89%, with melanoma at 79.13% and basal cell carcinoma at 82.88%. The work highlights the effectiveness of transfer learning and fine-tuning for high-performance multi-class skin lesion classification.

Chaturvedi et al. [20] uses MobileNet with transfer learning on the HAM10000 dataset for seven-class skin cancer classification. The model achieves 83.1% accuracy, with top-2 and top-3 accuracies of 91.36% and 95.34%. Weighted precision, recall, and F1-score are ~0.89, ~0.83, and ~0.83, demonstrating effective multi-class classification with a lightweight CNN.

Chapter 2

UNDERSTANDING MLOPS, RISKS AND REQUIREMENT ANALYSIS

2.1 Understanding MLOps, Risks and Requirement analysis

In this project, MLOps is used to manage the complete lifecycle of the skin disease classification system, ensuring that the model development process is structured, reproducible, and reliable. Since the project involves medical image classification, it is important to maintain consistent performance, proper version control, and continuous monitoring.

MLOps enables the following in this project:

- Data management: Handling image datasets, preprocessing steps, and dataset versioning.
- Model training and evaluation: Training CNN-based models and traditional machine learning classifiers in a standardized pipeline.
- Experiment tracking: Logging model parameters, evaluation metrics, and outputs using ZenML.
- Model versioning and deployment: Managing different versions of trained models and deploying only validated models.
- Monitoring and maintenance: Observing model behavior after deployment and identifying performance degradation.

Requirement Analysis

The requirements of the skin disease classification project are as follows:

Functional Requirements

- Accurate classification of multiple skin disease categories from input images.
- Support for multiple model configurations and architectures.
- Automated training and evaluation pipelines.
- Storage of experiment results and model versions.

Non-Functional Requirements

- High reliability and consistency of predictions.
- Scalability for larger datasets.
- Reproducibility of experiments.
- Secure handling of data and models.

Identifying these requirements helps in anticipating risks that may arise during model development and deployment.

2.2. Risk Matrix:

Risks identified:

- **Poor Generalization:** Due to variations in skin tone, lighting conditions, and image quality, the model may fail to generalize well to unseen clinical images.

  Impact: Incorrect classification on real-world images.
  Mitigation: Data augmentation, cross-validation, and balanced dataset usage.

- **Data Risk:** Skin disease datasets may contain class imbalance, noisy labels, or limited samples for certain diseases, which can significantly affect model performance.

  Impact: Biased or unreliable predictions.
  Mitigation: Data validation pipelines, dataset versioning, and preprocessing checks using ZenML.

- **Performance drift risk:** The distribution of input images may change over time due to new disease patterns or different imaging devices.

  Impact: Gradual reduction in classification accuracy.
  Mitigation: Continuous evaluation, retraining pipelines, and drift monitoring.

- **Model Overfitting:** Deep CNN models may memorize training data instead of learning meaningful features, especially with limited datasets.

  Impact: High training accuracy but poor validation/test performance.
  Mitigation: Regularization, early stopping, dropout layers, and proper train-validation splits.

- **Explainability Risk:** Medical practitioners require transparency in predictions, but CNN models are often black boxes.

  Impact: Reduced trust in the system.
  Mitigation: Grad-CAM visualizations to highlight affected skin regions.

- **Ethical and Security Risk:** Since the project involves medical data, issues such as data privacy, misuse of predictions, and model security must be considered.

  Impact: Legal, ethical, and trust concerns.
  Mitigation: Secure data handling, restricted access, and ethical usage guidelines.

Likelihood	Negligible Risk	Low Risk	High Risk	Severe Risk
Very likely	-	-	Poor Generalization	Data risk
Likely	-	-	Performance drift risk	Model overfitting
Unlikely	-	-	Explainability Risk	Ethical and security risk
Very unlikely	-	-	-	-

Role of risk matrix:

The risk matrix helps:

- Prioritize high-impact risks such as data quality and overfitting
- Align MLOps practices with medical application requirements
- Improve robustness and trustworthiness of the deployed model
- Support responsible and sustainable AI development

Chapter 3

DATA UNDERSTANDING, FEATURE ENGINEERING AND GOVERNANCE

3.1 Data Collection:

The dataset used in this study was obtained from a publicly available Kaggle repository: https://www.kaggle.com/datasets/ismailpromus/skin-diseases-image-dataset

This dataset contains dermatological images corresponding to 10 different skin disease categories, each with a varying number of samples, reflecting real-world data imbalance. The diseases included in the original dataset are Eczema, Warts, Molluscum, and other Viral Infections, Melanoma, Atopic Dermatitis, Basal Cell Carcinoma, Melanocytic Nevi, Psoriasis, Lichen Planus, and related diseases, Seborrheic Keratoses, Tinea / Ringworm / Fungal Infections and Additional benign and infectious dermatological conditions.

Each class contains a different number of images, capturing variability in disease prevalence and visual patterns. From this dataset, a subset of clinically relevant classes was later selected and balanced to support fair model training and evaluation.

3.2 Data Profiling, Cleaning, and Preprocessing:

From the original dermatology dataset—which contains images of multiple skin diseases captured from different body parts—five representative skin disease classes were selected: Atopic Dermatitis, Eczema, Benign Keratosis-like Lesions (BKL), Tinea/Ringworm/Fungal Infections, and Seborrheic Keratoses and other Benign Tumors. These classes were randomly selected to ensure that the resulting subset retained visual diversity across various anatomical regions, thereby preventing bias toward specific body locations. Random selection also helps improve model generalization by exposing the learning process to heterogeneous skin textures and lighting conditions.

To reduce class imbalance and maintain uniform representation, 1,250 images per class were randomly sampled. All images were resized to 224×224 pixels to standardize input dimensions. Pixel intensities were normalized to the [0,1] range to stabilize gradient updates during training.

To further enhance robustness and reduce overfitting, extensive data augmentation was applied to the training set. This included geometric transformations such as horizontal flipping, random rotations, zooming, width and height shifts, and shear transformations. In addition, custom augmentation techniques were introduced, including random Gaussian blurring, brightness adjustment, and contrast variation, simulating real-world image acquisition variability such as camera focus changes, illumination differences, and skin texture noise.

3.3 Feature Extraction and Feature Selection:

Feature extraction was performed using a custom deep Convolutional Neural Network (CNN) designed to learn hierarchical and discriminative representations from skin images. The architecture consists of four convolutional blocks with increasing filter depths (32, 64, 128, and 256), enabling progressive learning of low-level features (edges and color gradients) to high-level semantic patterns (lesion shapes and textures). Each block integrates Batch Normalization to stabilize learning, Max Pooling for spatial down-sampling, and Dropout regularization to prevent overfitting.

Following the convolutional layers, Global Average Pooling is employed to reduce spatial dimensions while preserving salient feature information. A fully connected dense layer with 256 neurons further refines the learned representations. The output of this dense layer (prior to classification) is designated as the feature embedding, producing a compact yet informative feature vector for each image.

These extracted deep features serve as the selected feature set, as they capture the most relevant visual characteristics required for skin disease discrimination. The intermediate feature vectors can be used for traditional machine learning classifiers.

3.4 Model Design Approaches:

The proposed system follows a hybrid modeling strategy that combines CNN-based deep feature extraction with traditional machine learning classifiers. Initially, the extracted feature vectors were used to train Logistic Regression, Random Forest, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM) models individually, and their respective performance metrics were evaluated.

A comparative evaluation table is provided below to analyze the strengths and limitations of each classifier.

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**
Logistic Regression	92.49%	92.51%	92.49%	92.49%
Random Forest	93.15%	93.24%	93.15%	93.15%
K-Nearest Neighbour	92.01%	92.22%	92.01%	92.02%

Support Vector Machine	90.69%	90.78%	90.69%	90.69%
Ensemble	93.34%	93.35%	93.33%	93.32%

Since no single classifier consistently outperformed the others across all classes, an ensemble learning approach was adopted. The final model integrates all four classifiers using a majority voting mechanism, leveraging their complementary decision boundaries. The ensemble was evaluated on a balanced test set of 1,666 images spanning five major skin conditions and achieved a testing accuracy of 93.34%, with precision, recall, and F1-score all at 93.3%, and an AUC score of 0.988, indicating strong discriminative capability. While Benign Keratosis-like lesions achieved perfect classification scores, slightly lower recall for Tinea and Eczema reflects the inherent visual similarity between these conditions, though overall performance remained robust and reliable.

3.5 Governance and Data Version Control:

To ensure reproducibility, traceability, and proper data governance, Git was used for version control of source code, configuration files, and experiment scripts, while Data Version Control (DVC) was employed to manage large datasets and derived artifacts. DVC enables Git-like tracking of dataset versions without storing bulky image files directly in the repository, ensuring efficient collaboration and experiment reproducibility. Together, Git and DVC provide a structured audit trail for dataset evolution, preprocessing changes, and model experimentation across development cycles.

Chapter 4

BUILDING AND EVALUATING AN ML MODEL

4.1 Model Building:

The model architecture follows a hybrid design combining deep feature extraction with traditional machine learning classifiers. A custom Convolutional Neural Network (CNN) is constructed with an input size of 224 × 224 × 3. The network consists of multiple convolutional blocks with filter sizes of 32, 64, 128, and 256, each followed by batch normalization, max pooling, and dropout layers to reduce overfitting. After the final convolutional block, a Global Average Pooling layer is applied to reduce spatial dimensions while retaining semantic information.

The pooled features are passed through a fully connected dense layer of 256 neurons, producing a 256-dimensional feature vector that serves as a compact and discriminative representation of each image. This penultimate layer output is used as the CNN feature extractor. A final softmax layer is used only during CNN training for supervised learning. The extracted 256-dimensional feature vectors are later used as input to Logistic Regression, Random Forest, SVM, and KNN classifiers, both individually and in an ensemble configuration.

4.2 Training, Validation and Testing:

The dataset is divided into training (70%), validation (15%), and testing (15%) subsets to enable systematic learning and performance monitoring. The CNN feature extractor is trained using the training set, while the validation set is used to monitor convergence and prevent overfitting. Training is configured with a maximum of 150 epochs, although early convergence is encouraged through callback mechanisms.

Early Stopping is applied by monitoring validation AUC with a patience of 20 epochs, ensuring that training halts when performance no longer improves while restoring the best weights. Additionally, a ReduceLROnPlateau scheduler dynamically reduces the learning rate by a factor of 0.5 if validation AUC stagnates for 3 epochs, with a minimum learning rate of 1×10^{-6}. This strategy stabilizes training and improves feature learning quality.

4.3 Evaluation Metrics and Visualization:

Model performance is evaluated using multiple standard classification metrics, including accuracy, precision, recall, F1-score, and AUC. The final ensemble model achieved an overall testing accuracy of 93.34%, with precision, recall, and F1-score all at approximately 93.3%, indicating strong and consistent classification performance across classes. A high AUC score of 0.988 further confirms the model's discriminative capability.

Figure 1 shows the classification report of the ensemble model.

```
Training Accuracy: 0.9693785649954968
Testing Accuracy : 0.9333733493397359

Confusion Matrix:
[[315  11   0   8  14]
[  3 231   0   0   3]
[  1   0 383   0   0]
[ 12   7   0 319  11]
[ 18   6   0  17 307]]

Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.91      0.90       348
           1       0.91      0.97      0.94       237
           2       1.00      1.00      1.00       384
           3       0.93      0.91      0.92       349
           4       0.92      0.88      0.90       348

    accuracy                           0.93      1666
   macro avg       0.93      0.93      0.93      1666
weighted avg       0.93      0.93      0.93      1666
```

Fig 1. Classification Report

To analyze class-wise behavior and misclassification patterns, a confusion matrix is computed and presented below in Figure 2.

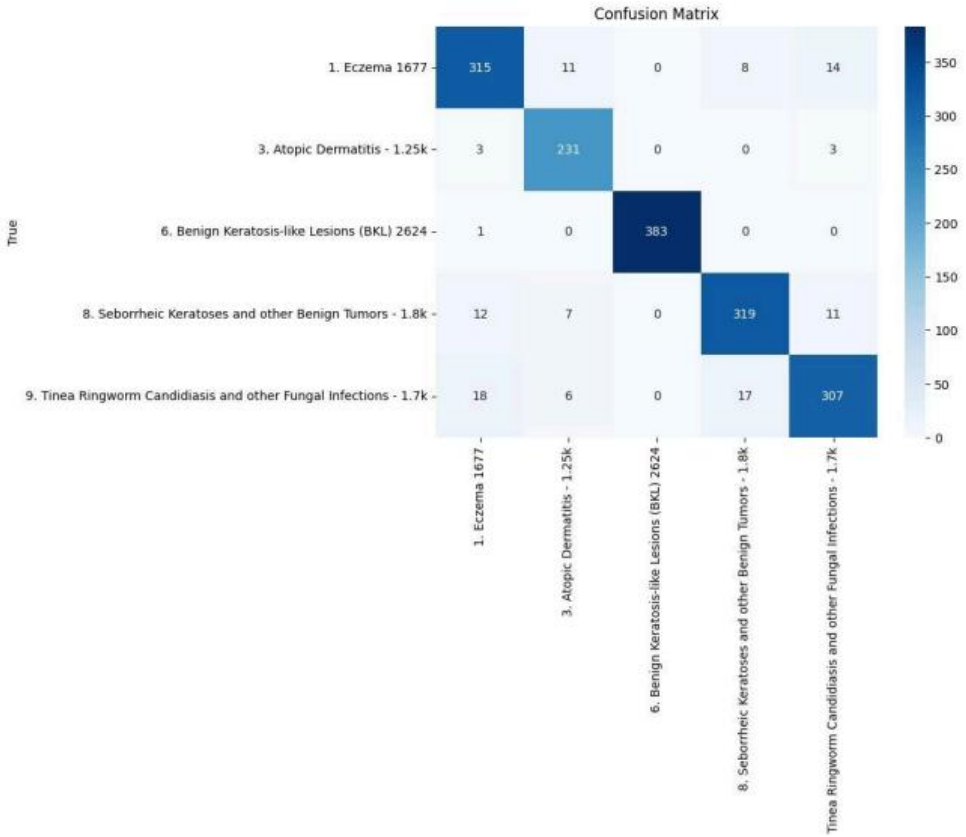

Fig 2. Confusion Matrix

This visualization highlights inter-class confusion, particularly among visually similar skin conditions, and supports a deeper understanding of model strengths and limitations.

4.4 Experiment Tracking:

Experiment tracking and pipeline orchestration are managed using ZenML, which provides structured tracking of experiments, parameters, metrics, and model versions across the ML lifecycle. ZenML pipelines ensure reproducibility by associating each experiment run with its corresponding dataset version and code state.

For artifact storage, MinIO is used as the backend to store trained models, extracted feature files, and intermediate outputs. This separation of compute and storage enables scalable experiment management and aligns with MLOps best practices by ensuring that all artifacts are versioned, traceable, and recoverable.

4.5 Interpreting Model Results and Error Analysis:

Detailed analysis of the classification results reveals that the ensemble model performs reliably across all skin disease categories. Certain classes, such as Benign Keratosis-like Lesions, achieved near-perfect classification scores due to distinct visual features. Slightly lower recall values observed for Eczema and Tinea can be attributed to visual similarity and overlapping texture patterns. These observations are supported by confusion matrix analysis and validate the effectiveness of the ensemble approach in mitigating individual classifier weaknesses.

Chapter 6

MODEL ANALYSIS AND REFINEMENT

6.1 Revisiting Model Performance

After developing the skin disease prediction model, model performance is revisited before deploying it. This is to ensure that the model can generalize well and maintain acceptable performance on unseen data.

The metrics that are evaluated are:

- Accuracy
- Precision, Recall, and F1-score
- Confusion Matrix

We compare the metrics across different training runs to identify stability and consistency in performance.

6.2 Hyperparameter Tuning and optimization

Hyperparameter tuning is performed to optimize model performance while balancing training time and computational cost. Techniques such as:

- Grid Search
- Random Search
- Early Stopping

are used during training.

Model interpretability techniques help understand how input features influence predictions.

6.3 Interpretability

Approaches like Feature importance analysis, Visualization of prediction confidence are used. These techniques improve trustworthiness and support ethical deployment in healthcare-related systems.

6.4 Model Versioning and Registry

Model versioning is handled using:

- Git for code version control

- DVC for dataset and experiment versioning
- ZenML Model Registry for managing trained models

Each trained model is assigned a unique version, linked to the dataset version used in it and associated with metadata such as performance metrics and training parameters. This ensures reproducibility across model lifecycle.

6.5 Documentation of Model Updates

All updates to the model—including retraining, parameter changes, and dataset updates—are documented. This includes change logs, version history and reasons for retraining or modification.

Chapter 7

IMPLEMENTING CI/CD PIPELINE

7.1 Overview of CI/CD for ML

Continuous Integration and Continuous Deployment (CI/CD) in machine learning automates the process of:

- Training
- Testing
- Packaging
- Deploying models

Unlike traditional software CI/CD, ML pipelines must also handle data versioning, model validation, and monitoring feedback

7.2 Tools Setup

The CI/CD pipeline integrates the following tools:

- GitHub for source code management
- GitHub Actions for automated workflows
- ZenML for pipeline orchestration
- Docker for containerized deployment

Each code push or pull request triggers automated checks to ensure system stability.

7.3 Automating Model Training and Testing

Automated workflows are configured to:

- Fetch the correct dataset version using DVC
- Run training pipelines via ZenML
- Execute validation and basic sanity checks
- Store outputs and artifacts in MinIO

This reduces manual intervention and ensures consistency.

7.4 Building and Running a Deployment Pipeline

Once a model passes validation:

- A Docker image is built containing the inference code
- The image is tested locally or in staging
- The model is deployed as a containerized service

ZenML ensures that the deployed model corresponds exactly to a registered and approved model version.

7.5 Continuous Integration Hands-on Exercise

A sample CI workflow includes:

1. Code commit to GitHub
2. Automated pipeline trigger
3. Dataset pull via DVC
4. Model training and validation
5. Artifact storage in MinIO
6. Deployment readiness check

This demonstrates a complete MLOps lifecycle in practice.

Chapter 8

MONITORING AND TRACKING MODEL LIFECYCLE

8.1 Model Monitoring Concepts and KPIs

Post-deployment monitoring ensures the model continues to perform reliably in production. Key Performance Indicators (KPIs) that are monitored are:

- Prediction accuracy over time
- Input data distribution
- Prediction confidence
- Error rates

8.2 Monitoring Tools

The monitoring stack consists of:

- Evidently AI for model performance and drift detection
- Prometheus for metrics collection
- Grafana for visualization dashboards

Evidently AI generates reports for data drift, concept drift and prediction quality of the model.

8.3 Detecting Concept Drift and Data Drift

Data drift occurs when incoming data differs from training data. Concept drift occurs when the relationship between inputs and outputs changes.

Evidently AI automatically detects:

- Feature distribution shifts
- Changes in prediction behavior

Alerts are triggered when drift exceeds predefined thresholds.

8.4 Model retraining and redeployment

When drift or performance degradation is detected through monitoring tools such as Evidently AI, the system initiates a controlled response to maintain model reliability. Retraining pipelines are triggered using the latest available data, ensuring that the model adapts to changes in input distributions or underlying patterns. The newly trained models then undergo validation against predefined performance thresholds to confirm that they meet quality and stability requirements. Once validated, these models are registered as new versions in the model registry, preserving full

traceability with respect to data, parameters, and metrics. Finally, the approved model version is deployed to production using automated deployment pipelines, replacing the older version in a seamless manner. This closed feedback loop enables continuous learning and adaptation while ensuring consistency, reproducibility, and minimal service disruption.

8.5 Logging, Alerting and Dashboard Setup

Logs include:

- Prediction logs
- Error logs
- System health metrics

Dashboards provide real-time visibility into model health and system performance.

Chapter 9

PERFORMANCE EVALUATION AND GOVERNANCE

9.1 Post-Deployment Performance Review

After deployment, the model's performance is continuously evaluated using real-world inference data to ensure it behaves as expected outside the training environment. Key metrics such as accuracy, error rate, and prediction confidence are compared against baseline values obtained during validation. Any discrepancy between offline (training/validation) performance and online (production) performance is carefully analyzed. Performance reports generated by monitoring tools help identify early signs of degradation or instability. These reviews are conducted periodically to ensure long-term reliability of the deployed model.

9.2 Continuous Improvement via feedback loops

Feedback from monitoring systems, user interactions, and prediction outcomes is systematically collected and analyzed.

- Data collection: Detected issues such as recurring misclassifications or confidence drops inform future data collection and preprocessing strategies.
- Model retraining: Monitoring insights are fed back into retraining pipelines to improve model robustness and generalization.

This feedback loop ensures that the model evolves in response to real-world usage rather than remaining static. Automated pipelines help shorten the iteration cycle while maintaining consistency.

9.3 Governance and Ethical AI Practices

Given the medical nature of the application, ethical considerations include:

- Bias minimization: Measures are taken to reduce bias by ensuring diverse and representative training data.
- Transparency in predictions: Model decisions are made transparent through interpretability techniques and documented limitations.

- Responsible use of patient data: User data privacy and security are prioritized, and sensitive information is handled responsibly.
- Clear limitations of model usage: Clear disclaimers are provided stating that the system is a decision-support tool and not a replacement for professional medical diagnosis.

9.4 Documentation: Model Cards and Data Cards

Model Cards are created to document essential information such as intended use, model performance metrics, training data characteristics, and known limitations. They also include ethical considerations and scenarios where the model should not be used. Data Cards describe the dataset source, preprocessing steps, data quality checks, and licensing terms. Together, these documents improve transparency, reproducibility, and accountability across the model lifecycle. Proper documentation also aids in audits, maintenance, and future enhancements.

9.5 Audit and Review Checklist

The system is periodically reviewed for:

- Compliance: Regular audits are conducted to verify compliance with performance, security, and governance standards.
- Performance stability: Monitoring logs and alert histories are reviewed to ensure timely responses to drift or failures and help ensure model stability.
- Ethical adherence: Ethical and regulatory considerations are reassessed periodically as data and usage contexts evolve.

Ethical and regulatory considerations are reassessed periodically as data and usage contexts evolve.

Chapter 10

APPENDICES

A.1 List of MLOps Tools used

Tool	Purpose	Pros	Cons
Git	Code versioning	Widely used, reliable	No data tracking
DVC	Data versioning	Reproducibility	Initial setup complexity
ZenML	Pipeline automation	Modular, scalable	Learning curve
MinIO	Artifact storage	S3-compatible, fast	Needs configuration
Docker	Deployment	Portable, consistent	Resource overhead
Evidently AI	Monitoring	Drift detection	Requires reference data

A.2 Tool Installation Guides

Git

- Install Git from the official Git website based on the operating system.
- Verify installation using `git --version`.
- Configure username and email for version control using `git config`.
- Git is used for source code management and collaboration.

DVC (Data Version Control)

- Install DVC using `pip install dvc` or OS-specific package managers.
- Initialize DVC in the project repository using `dvc init`.
- Configure remote storage (local or cloud) for dataset tracking.
- DVC enables dataset and experiment versioning alongside Git.

ZenML

- Install ZenML using `pip install zenml`.
- Initialize ZenML in the project using `zenml init`.
- Install required integrations such as Docker and ML frameworks.
- Register and configure stacks to manage pipelines, artifact stores, and orchestration.

Docker

- Install Docker Desktop for Windows/Linux/macOS.
- Verify installation using `docker --version.`
- Enable Docker daemon and ensure it runs in the background.
- Docker is used to containerize the application for consistent deployment.

MinIO

- Install MinIO server using Docker or standalone binaries.
- Start the MinIO service and configure access keys.
- Register MinIO as an artifact store in ZenML.
- MinIO stores model artifacts, pipeline outputs, and metadata.

A.3 Dataset References and Licensing

The dataset used in this study was obtained from a publicly available Kaggle repository: https://www.kaggle.com/datasets/ismailpromus/skin-diseases-image-dataset

It is distributed under the Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0) license, which allows use and adaptation for academic research with appropriate attribution, but does not permit commercial use without explicit permission.

A.5 Folder Structure

```
mlops-skin-disease/
│
├── data/
│   ├── raw/
│   ├── processed/
│
├── pipelines/
├── models/
├── monitoring/
├── docker/
├── reports/
├── dvc.yaml
├── zenml_pipeline.py
└── README.md
```

A.6 Reference Papers and Further Reading

[1]Alrahal, M. K., & Astour, S. (2025). "*MLOps: Changes and tools – Adapting machine learning workflows for enhanced efficiency and scalability"*. *International Journal of Engineering Research & Technology (IJERT), 14*(10), Article IJERTV14IS100158. No DOI available.

[2]B. Eken, S. Pallewatta, N. K. Tran, A. Tosun, and M. A. Babar, *"A multivocal review of MLOps practices, challenges and open issues,"* arXiv preprint arXiv:2406.09737, Jun. 2024, doi: 10.48550/arXiv.2406.09737.

[3]*"A Systematic Review of Machine Learning, Deep Learning, and Transfer Learning Methods for Skin Disease Classification"*, Int. J. Environ. Sci., pp. 1152–1175, Aug. 2025, doi: 10.64252/pqr4j094.

[4]*"Enhancing skin disease classification leveraging transformer-based deep learning architectures and explainable AI," arXiv preprint arXiv:2407.14757, Jul. 2024, doi: 10.48550/arXiv.2407.14757.*

[5]*"Machine learning operations landscape: Platforms and tools,"* Artif. Intell. Rev., vol. 58, Art. no. 167, 2025, doi: 10.1007/s10462-025-11164-3.

[6] F. Amou Najafabadi, J. Bogner, I. Gerostathopoulos, and P. Lago, *"An analysis of MLOps architectures: A systematic mapping study,"* arXiv preprint arXiv:2406.19847, Jun. 2024, doi: 10.48550/arXiv.2406.19847.

[7] G. Mallardi, F. Calefato, L. Quaranta and F. Lanubile, *"An MLOps Approach for Deploying Machine Learning Models in Healthcare Systems,"* 2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), Lisbon, Portugal, 2024, pp. 6832-6837, doi: 10.1109/BIBM62325.2024.10822603.

[8] Furqon, Ainul & Malik, Kamil & Fajri, Fathorazi. (2024). Detection of Eight Skin Diseases Using Convolutional Neural Network with MobileNetV2 Architecture for Identification and Treatment Recommendation on Android Application. Jurnal Ilmiah Teknik Elektro Komputer dan Informatika. 10. 373-384. 10.26555/jiteki.v10i2.28817.

[9] Yamachui Sitcheu, Angelo & Friederich, Nils & Baeuerle, Simon & Neumann, Oliver & Reischl, Markus & Mikut, Ralf. (2023). *MLOps for Scarce Image Data: A Use Case in Microscopic Image Analysis.* 10.48550/arXiv.2309.15521.

[10] Rifat Sadik, Anup Majumder, Al Amin Biswas, Bulbul Ahammad, Md. Mahfujur Rahman, An in-depth analysis of Convolutional Neural Network architectures with transfer learning for skin disease diagnosis, Healthcare Analytics, Volume 3, 2023, 100143, ISSN 2772-4425, https://doi.org/10.1016/j.health.2023.100143.

[11] R. Agarwal and D. Godavarthi, "Skin Disease Classification Using CNN Algorithms", *EAI Endorsed Trans Perv Health Tech*, vol. 9, Oct. 2023, doi: 10.4108/eetpht.9.4039.

[12] Patel, Avni & Parmar, Pravina & Thakkar, Nilam & Barot, Hitesh. (2023). *Skin Disease Classification Using Machine Learning Algorithms: A Review.* Doi not available

[13] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine Learning Operations (MLOps): Overview, Definition, and Architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023, doi: 10.1109/ACCESS.2023.3262138.

[14] Srujan, S. A., Shetty, C. M., Adil, M., Sarang, P. K., & Roopitha, C. H. (2022). *Skin Disease Detection using Convolutional Neural Network*. *International Research Journal of Engineering and Technology (IRJET)*, 09(07), 2162-2165. Retrieved from https://www.irjet.net/archives/V9/i7/IRJET-V9I7397.pdf

[15] D. V. Vasudha Rani, G. Vasavi and B. Maram, "Skin Disease Classification Using Machine Learning and Data Mining Algorithms," 2022 IEEE 2nd International Symposium on Sustainable Energy, Signal Processing and Cyber Security (iSSSC), Gunupur, Odisha, India, 2022, pp. 1-6, doi: https://doi.org/10.22214/ijraset.2024.59286

[16] N. Hewage and D. Meedeniya, *"Machine learning operations: A survey on MLOps tool support,"* arXiv preprint, Feb. 2022.

[17] P. N. Srinivasu, J. G. S. Sai, M. F. Ijaz, A. K. Bhoi, W. Kim, and J. J. Kang, *"Classification of skin disease using deep learning neural networks with MobileNet V2 and LSTM,"* Sensors, vol. 21, no. 8, Art. no. 2852, Apr. 2021, doi: 10.3390/s21082852.

[18] M. A. Kassem, K. M. Hosny, R. Damaševičius, and M. M. Eltoukhy, *"Machine learning and deep learning methods for skin lesion classification and diagnosis: A systematic review,"* Diagnostics, vol. 11, no. 8, Art. no. 1390, Jul. 2021, doi: 10.3390/diagnostics11081390.

[19] P. Naronglerdrit and I. Mporas, *"Evaluation of big data based CNN models in classification of skin lesions with melanoma,"* arXiv preprint arXiv:2007.05446, Jul. 2020, doi: 10.48550/arXiv.2007.05446.

[20] S. S. Chaturvedi, K. Gupta, and P. S. Prasad, *"Skin lesion analyser: An efficient seven-way multi-class skin cancer classification using MobileNet,"* in Advanced Machine Learning Technologies and Applications (AMLTA 2020), Advances in Intelligent Systems and Computing, vol. 1141, Springer, Singapore, 2020, doi: 10.1007/978-981-15-3383-9_15.